Aerospace Science and Technology 133 (2023) 108091

Contents lists available at ScienceDirect

Aerospace Science and Technology

journal homepage: www.elsevier.com/locate/aescte

PASCAL: PopulAtion-Specific Curriculum-based MADRL for collision-free flocking with large-scale fixed-wing UAV swarms ☆

Chao Yan [a], Xiaojia Xiang [a], Chang Wang [a,*], Fei Li [b], Xiangke Wang [a], Xin Xu [a], Lincheng Shen [a]

[a] College of Intelligence Science and Technology, National University of Defense Technology, Changsha 410073, China
[b] Computer Network Information Center, Chinese Academy of Sciences, Beijing 100083, China

ARTICLE INFO

Article history:
Received 26 September 2022
Received in revised form 14 December 2022
Accepted 23 December 2022
Available online 2 January 2023
Communicated by Xiande Fang

Keywords:
UAV swarm
Flocking
Collision avoidance
Deep reinforcement learning
Curriculum learning

ABSTRACT

Flocking with a swarm of unmanned aerial vehicles (UAVs) has been playing an important role in various applications. However, the complexity of developing a collision-free flocking policy for a UAV swarm increases tremendously with the swarm scale. In this paper, a novel curriculum-based multi-agent deep reinforcement learning (MADRL) approach is proposed for flocking with collision avoidance of large-scale fixed-wing UAV swarms by progressively increasing the population of UAVs, which we call **P**opul**A**tion-**S**pecific **C**urriculum-based M**A**DR**L** (PASCAL). Specifically, PASCAL has two core components of policy learning and knowledge transfer. For policy learning, an improved multi-agent deep deterministic policy gradient algorithm is proposed to accelerate the learning process. For knowledge transfer, an attention-based population-invariant network is designed to handle dynamic dimensional inputs, enabling the reloading and fine-tuning of the previously learned models. Finally, both numerical and semi-physical simulations demonstrate the advantages of PASCAL in terms of learning efficiency and generalization capability for different swarm scales.

© 2022 Elsevier Masson SAS. All rights reserved.

1. Introduction

Over the past decade, unmanned aerial vehicles (UAVs) have been widely applied in various fields, such as area monitoring, urban air mobility, and relay communication [1–3]. Compared with a single UAV, a swarm of UAVs can solve a given task in a more effective and efficient way due to the advantages of robustness, survivability, and redundancy [4]. Therefore, the cooperation of a UAV swarm has attracted extensive attention [5], where flocking with collision avoidance is a fundamental control problem [6]. In recent years, large-scale UAV swarms have shown great potential in efficiently completing difficult and complex tasks [7,8]. However, developing a collision-free flocking policy for a scalable and large-scale UAV swarm remains challenging due to task complexity, environmental dynamics, and limited sensing capacity.

Flocking is a collective phenomenon that has been widely observed in nature [9]. This biological behavior is believed to be useful for the control of a robotic swarm, hence the flocking control problem has attracted extensive attention [10,11]. It is meaningful to develop an effective flocking control method for the civil and military applications of multiple UAVs, such as surveillance, mapping and surveying, search and rescue. For example, the leader-follower flocking model can be used for manned-unmanned aerial vehicle cooperative operations, where the manned aerial vehicle is the leader and the UAVs are the followers.

Many researchers have developed different model-based methods for flocking with multiple UAVs. In [12], a real-time flocking algorithm based on the consensus theory was proposed for maintaining safe distances among the quadrotor UAVs. In [13], a flocking algorithm with shared obstacle information was proposed for fixed-wing UAVs, whose kinematics were modeled as second-order integrators. In [14], a distributed flocking control strategy was developed based on model predictive control, enabling a team of quadrotors to track the desired trajectory without collisions. These methods highly rely on accurate models of UAV kinematics and environmental disturbances to design control laws. However, iden-

☆ This work was supported in part by the Science and Technology Innovation 2030-Key Project of New Generation Artificial Intelligence under Grant 2020AAA0108200, in part by the National Natural Science Foundation of China under Grant 61825305, Grant 61906203, and Grant 81973244, in part by the Postgraduate Scientific Research Innovation Project of Hunan Province under Grant CX20210001.

* Corresponding author.
E-mail addresses: yanchao17@nudt.edu.cn (C. Yan), xiangxiaojia@nudt.edu.cn (X. Xiang), wangchang07@nudt.edu.cn (C. Wang), pittacus@gmail.com (F. Li), xkwang@nudt.edu.cn (X. Wang), xinxu@nudt.edu.cn (X. Xu), lcshen@nudt.edu.cn (L. Shen).

https://doi.org/10.1016/j.ast.2022.108091
1270-9638/© 2022 Elsevier Masson SAS. All rights reserved.

tifying an accurate model is challenging and even unavailable in real-world applications.

Alternatively, model-free deep reinforcement learning (DRL) [15] that combines the advantages of deep learning and reinforcement learning (RL) has been proved effective in a wide range of tasks, such as path planning [16], coordinated control [17], and rendezvous guidance [18]. Some researchers have successfully applied DRL to flocking control of rotary UAVs. In [19], the deep deterministic policy gradient [20] algorithm was used to deal with the flocking and navigation problem for quadrotor UAVs. In [21], the proximal policy optimization [22] algorithm was used to train a shared flocking policy in an environment with dense obstacles. The above work demonstrated that DRL was effective for flocking control with rotary UAVs. However, these methods assumed the UAVs were particle-based agents for simplification, which was not applicable to fixed-wing UAVs due to their more complex kinematic models [23].

Little effort has been devoted to designing a flocking controller for fixed-wing UAVs in the context of RL. Hung *et al.* [24,25] were the first to apply RL for flocking with fixed-wing UAVs in stochastic environments. They proposed two algorithms of Dyna-Q-Flocking [24] and Q-Flocking [25], taking the model-based Dyna [26] approach and the model-free Q-learning [26] approach, respectively. Speck *et al.* [27] presented a behavioral flocking controller for fixed-wing UAVs based on SARSA [26] and object-focused learning. The above methods solved the flocking problem for fixed-wing UAVs preliminarily by discretizing the state and action spaces and assuming the UAVs had the same constant speed for simplification. However, these assumptions are not applicable to real-world UAV tasks where the states and actions of UAVs are continuous, and the speed is allowed to change as needed.

In our previous work [28], we formulated the fixed-wing UAV flocking problem in the same way as in [25] and proposed a DRL-based algorithm to train an end-to-end policy, controlling the roll angles and speed of fixed-wing UAVs in the continuous action space. Consistent with [24,25,27], we also assumed that the UAVs flied at different heights to ignore the collision avoidance problem in [28]. Nevertheless, some applications require the UAVs to fly at the same height to maintain a specific formation, which means the UAVs have the risk of collision if they are too close. In our recent work [29], we solved the collision avoidance problem by designing a local situation map for each UAV to describe its collision risks with the other UAVs nearby. The dimension of the local situation map is independent of the number of the neighbor-followers, endowing scalability to the learned flocking policy. However, constructing local situation maps for all the UAVs is computationally expensive, especially when the number of UAVs increases dramatically.

For large-scale fixed-wing UAV swarms, it is inappropriate to directly employ existing DRL algorithms to learn collision-free flocking policies. First, the dimension of the state and action spaces increases exponentially as the number of UAVs increases [30], resulting in the infamous curse of dimensionality problem. Besides, the system dynamics and environmental uncertainties become even more complex considering the interactions among UAVs [8]. As a result, it may require a long training time, or fail to learn a high-quality flocking policy.

The curriculum learning paradigm [31] is one possible solution for the above problem. It treats a complex task as a series of simpler tasks in a staged manner, and the training process is started from easy conditions that gradually become more difficult. For example, the multi-agent motion planning problem was solved in four stages that the number of agents was increased in each stage [32]. The dynamic multi-agent curriculum learning paradigm was proposed to solve complex cooperative problems with a large number of agents [33]. Then, the evolutionary population curriculum learning framework further integrated the insight of evolutionary learning to solve large-scale multi-agent RL problems [34]. These studies show that leveraging curriculum learning over the agent population can not only significantly reduce convergence time but also improve policy performance.

Inspired by the above research, this paper integrates curriculum learning into multi-agent DRL (MADRL) to deal with the flocking and collision avoidance problem for a large-scale fixed-wing UAV swarm. The key insight is to divide the training process into multiple learning stages with a increasing number of UAVs. We start to learn the policy with a small-scale UAV swarm, and then progressively expand the scale of the UAV swarm until the desired scale, and finally finish the curriculum. The main contributions are as follows:

1. We develop a **P**opul**A**tion-**S**pecific **C**urriculum-based M**A**DR**L** (PASCAL) approach for collision-free flocking with a large-scale UAV swarm. By progressively expanding the population of UAVs and alternatively undergoing the process of policy learning and knowledge transfer in a staged manner, PASCAL shows significant advantages in policy optimality and learning efficiency.
2. We propose an improved multi-agent deep deterministic policy gradient algorithm based on the parameter sharing with ego-centric representation mechanism to accelerate the learning process among the UAV swarm in each learning stage.
3. We design an attention-based population-invariant network structure leveraging the self-attention mechanism. The network structure can not only adapt to dynamic dimensional input, but also allow the reloading and fine-tuning of the previously learned models.

The rest of this paper is organized as follows. Section 2 formulates the flocking control with collision avoidance problem in the context of MADRL. Section 3 proposes the curriculum-based MADRL approach, followed by experiments and results in Section 4. Finally, Section 5 concludes this paper and outlines our plans for future work.

2. Problem formulation

This section describes the flocking with collision avoidance problem for large-scale fixed-wing UAV swarms. Then, we model this problem as a decentralized partially observable Markov decision process (Dec-POMDP) [35] in the context of MADRL.

2.1. Problem statement

As shown in Fig. 1, in our flocking scenario, a dynamic homogeneous swarm of fixed-wing UAVs flies at a fixed altitude with the leader-follower topology. Specifically, only one leader-UAV (leader) is followed by a variable number of follower-UAVs (followers). The leader's behavior is determined by the external commands generated by the operator or the ground control station based on its specific tasks. We focus on the design of the collision-free flocking policy for the followers. The followers are also called agents in the sequel.

The objective for each follower is to maintain a certain distance from the leader and avoid collisions with other followers at the same time. Thus, we assume that each follower is able to obtain the leader's state, as well as the states of the other followers within its maximum sensing range (R_c) (termed as *neighbors*) via the wireless communication channel or the airborne radar. In the following, we will propose a curriculum-based MADRL approach to train the followers to independently select their steering commands according to the states of the leader and their neighbors,

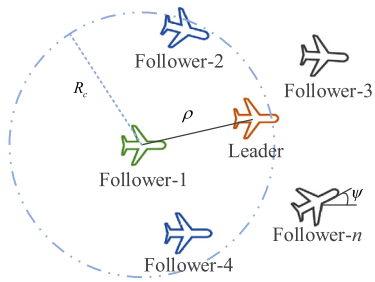


Fig. 1. A scalable fixed-wing UAV swarm consisting of one unique leader and n identical followers. Each follower determines its steering commands to flock with the leader and avoid collision with the others.

so as to flock with the leader and avoid conflicts with the other followers simultaneously.

2.2. Dec-POMDP of collision-free flocking

The collision-free flocking problem can be modeled as a Dec-POMDP. Its main ingredients are defined as follows.

2.2.1. State representation

Given the assumption that the UAV swarm flies at a fixed altitude, the state of an individual UAV can be represented by a tuple defined as $\xi := (x, y, \psi, \phi, v)$, where (x, y) is the planar position of the UAV; ψ is the heading angle; ϕ is the roll angle; v is the airspeed. The global system state can be represented as $s := (\xi_l, \boldsymbol{\xi}_f)$. Here $\xi_l := (x_l, y_l, \psi_l \phi_l, v_l)$ denotes the leader's state, and $\boldsymbol{\xi}_f := \{\xi_f^i \mid i \in \mathcal{I}\}$ represents the followers' states, where $\xi_f^i := (x_f^i, y_f^i, \psi_f^i, \phi_f^i, v_f^i)$.

2.2.2. Observation space

The observation for each follower is limited by the maximum sensing range R_c. For each follower i, the observation $o_i := (\xi_f^i, \boldsymbol{\xi}_{\mathcal{N}_i}, \xi_l)$ consists of three parts: the ego state ξ_f^i, its neighbors' states $\boldsymbol{\xi}_{\mathcal{N}_i} = \{\xi_f^j \mid j \in \mathcal{N}_i\}$, and the leader's state ξ_l.

2.2.3. Action space

We control the followers by selecting the roll and velocity actions. For each follower i, we define the action $a_i := \left(a_r^i, a_v^i\right)$ in continuous spaces, where $a_r^i \in (-a_{\max}^r, a_{\max}^r)$ is the roll action and $a_v^i \in (-a_{\max}^v, a_{\max}^v)$ is the velocity action. Given the current roll angle ϕ_i, the next roll angle setpoint ϕ_{ref}^i is defined by

$$\phi_{\text{ref}}^i = \begin{cases} r_{\text{bd}} & \text{if } \phi_i + a_r^i > r_{\text{bd}} \\ -r_{\text{bd}} & \text{if } \phi_i + a_r^i < -r_{\text{bd}} \\ \phi_i + a_r^i & \text{otherwise} \end{cases}, \tag{1}$$

where $[-r_{\text{bd}}, r_{\text{bd}}]$ is the allowed range of the roll angle. The next velocity setpoint v_{ref}^i can be calculated by

$$v_{\text{ref}}^i = \begin{cases} v_{\max} & \text{if } v_i + a_v^i > v_{\max} \\ v_{\min} & \text{if } v_i + a_v^i < v_{\min} \\ v_i + a_v^i & \text{otherwise} \end{cases}, \tag{2}$$

where v_i is the current velocity; $v_{\max}$ and $v_{\min}$ are the maximum and minimum velocities, respectively.

2.2.4. State transition model

Consistent with our previous work [28], we model the fixed-wing UAV kinematics with stochastic disturbances to simulate flight data for training the collision-free flocking policy in the context of MADRL. The established kinematics can be reformulated as

$$\dot{\xi} = \frac{d}{dt}\begin{Bmatrix} x \\ y \\ \psi \\ \phi \\ v \end{Bmatrix} = \begin{Bmatrix} v\cos\psi + \eta_x \\ v\sin\psi + \eta_y \\ -\left(\alpha_g / v\right)\tan\phi + \eta_\psi \\ f_\phi\left(\phi, \phi_{\text{ref}}\right) \\ f_v\left(v, v_{\text{ref}}\right) \end{Bmatrix}, \tag{3}$$

where α_g is the acceleration of gravity. The disturbance terms η_x, η_y, and η_ψ are drawn from the normal distributions $\mathbb{N}\left(\bar{\eta}_x, \sigma_x^2\right)$, $\mathbb{N}\left(\bar{\eta}_y, \sigma_y^2\right)$, and $\mathbb{N}\left(\bar{\eta}_\psi, \sigma_\psi^2\right)$, respectively. The function $f_\phi(\phi, \phi_{\text{ref}})$ and $f_v\left(v, v_{\text{ref}}\right)$ describe the roll and airspeed dynamics with uncertainties (see [28] for more details), respectively. We note that the established kinematic model is stochastic and uncertain.

2.2.5. Reward function

Our aim is to learn a collision-free flocking policy for large-scale UAV swarms to form a flock in a distributed manner. From the perspective of each follower, the flocking control with collision avoidance problem involves two objectives: maintaining a certain distance from the leader while avoiding collisions with its neighbors. For this purpose, we first specify the *cohesion reward* to encourage each follower to gather around the leader:

$$r_c^i = \begin{cases} -P & \rho_i \le R_s \\ \omega_1(\rho_i - R_a) & R_s < \rho_i \le R_a \\ -\rho_i - \omega_2|\Delta\psi_i| & \rho_i > R_a \end{cases}, \tag{4}$$

where ρ_i and $\Delta\psi_i$ denote the distance and the difference in the heading between the leader and the follower i, respectively; ω_1 and ω_2 are the tuning parameters; $P \gg 0$ is a large positive constant; R_s and R_a are the minimum safety distance and the alert radius of the follower, respectively.

In order to prevent each follower from colliding with its neighbor followers, the *separation reward* is designed as

$$r_s^{ij} = \begin{cases} -P & D_{ij} \le R_s \\ \omega_1(D_{ij} - R_a) & R_s < D_{ij} \le R_a \\ 0 & D_{ij} > R_a \end{cases}, \tag{5}$$

where D_{ij} represents the distance from the follower i to its j-th neighbor.

Finally, we define the reward function for each follower i as follows:

$$r_i = r_c^i + \sum\nolimits_{j \in \mathcal{N}_i} r_s^{ij}. \tag{6}$$

3. Population-specific curriculum-based MADRL approach

This section proposes a curriculum-based MADRL approach for collision-free flocking control of large-scale fixed-wing UAV swarms. Section 3.1 gives a brief overview of the proposed **P**opul**A**tion-**S**pecific **C**urriculum-based M**A**DR**L** (PASCAL) approach. Section 3.2 and Section 3.3 describe the details of the two core components of PASCAL, i.e., policy learning with the ego-centric representation based multi-agent deep deterministic policy gradient (ECR-MADDPG) algorithm and knowledge transfer with the attention-based population-invariant network (APINet).

3.1. PASCAL for large-scale UAV swarms

In multi-agent systems, the dimension of the state and action spaces grows exponentially with the number of agents [30]. Thus, it is challenging for large-scale fixed-wing UAV swarms to directly learn a collision-free flocking policy with good performance and high efficiency. To address this problem, we resort to the curriculum learning paradigm [31] to improve the performance of MADRL.

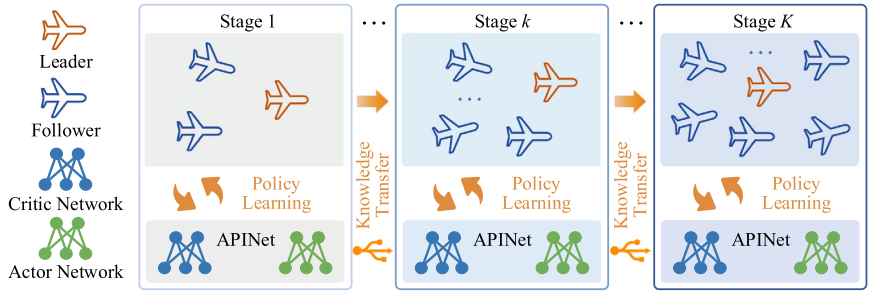


Fig. 2. Overview of the proposed PASCAL approach for flocking with a large-scale fixed-wing UAV swarm.

In curriculum learning, it is well known that how to tailor a curriculum properly is still an open question [31]. Considering the properties of UAV swarms, i.e., partial observability, sparse interactivity, and state semanticity [33], it is natural and reasonable to generate a curriculum according to the population of UAVs to solve the flocking with collision avoidance problem for large-scale UAV swarms. We refer to the designed curriculum as *population-specific curriculum*.

Based on the population-specific curriculum, we propose the PASCAL approach. As illustrated in Fig. 2, the key insight behind PASCAL is to divide the training process into multiple learning stages and progressively expand the scale of UAV swarms in the order of the stages. At each learning stage, the control objective and collision avoidance constraints are the same. Specifically, we start with a small-scale UAV swarm to learn the collision-free flocking control policy in the early stages, and then gradually expand the population of UAVs in the swarm in the later stages.

PASCAL consists of two iterative procedures: a policy learning procedure and a knowledge transfer procedure. During the former procedure, the agent optimizes the collision-free flocking policy under the guidance of the knowledge transferred from the previous stage. We will propose an efficient policy learning algorithm based on multi-agent deep deterministic policy gradient (MADDPG) [36] for this procedure in Section 3.2. The latter procedure transfers the knowledge learned in the previous stage to the next stage, which will be presented in Section 3.3.

3.2. Policy learning with ECR-MADDPG

Parameter sharing is an effective method to improve the scalability and learning efficiency of MADRL [37]. In our flocking scenario, the UAV swarm is homogeneous. We integrate the parameter sharing mechanism into the policy learning procedure, which means the followers are allowed to share the flocking policy with the same parameters. In the sequel, we first describe the ego-centric representation for parameter sharing. Then, we propose an improved MADDPG algorithm based on the parameter sharing with ego-centric representation mechanism [37], which is called ECR-MADDPG.

3.2.1. Ego-centric representation for parameter sharing

Although the parameters of the policy network (actor) for each follower are the same, it is still able to generate different behaviors among followers due to their unique observations. However, the input of the critic network is the global state, which is the same for all followers. In order to enable the shared critic network to output different Q-values among the followers, we re-represent the global system state for each follower from its own perspective. In other words, each follower selects itself as the reference to transform the system state s from the global coordinate frame to the ego-centric coordinate frame. We refer to this process as *ego-centric representation*.

Given the follower i's ego state ξ_f^i and the state of an arbitrary entity j in the global system state $\xi_j \in s$, the ego-centric system state for the follower i can be represented as

$$\tilde{s}_i := \{\tilde{\xi}_j \mid \tilde{\xi}_j = \mathcal{T}(\xi_j, \xi_f^i),\ \xi_j \in s\}, \tag{7}$$

where $\mathcal{T}(\cdot)$ denotes the transform function. Denote by $\xi_j := (x_j, y_j, \psi_j \phi_j, v_j)$ the state of entity j in the global coordinate frame, the transformed state can be defined as $\tilde{\xi}_j := (\tilde{x}_j, \tilde{y}_j, \tilde{\psi}_j, \phi_j, v_j)$, where

$$\begin{bmatrix} \tilde{x}_j \\ \tilde{y}_j \\ \tilde{\psi}_j \end{bmatrix} = \begin{bmatrix} \cos\psi_f^i & \sin\psi_f^i & 0 \\ -\sin\psi_f^i & \cos\psi_f^i & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_j - x_f^i \\ y_j - y_f^i \\ \psi_j - \psi_f^i \end{bmatrix}, \tag{8}$$

$(\tilde{x}_j, \tilde{y}_j)$ and $\tilde{\psi}_j$ represent the planar position and heading in the ego-centric coordinate frame, respectively.

Instead of concatenating the respective indices of the followers onto their respective observations, we also transform each follower's observation o_i from the global coordinate frame to the ego-centric coordinate frame. This transformation amplifies the differences between the observations of different followers. Similarly, the ego-centric observation for the follower i can be defined as

$$\tilde{o}_i := \{\tilde{\xi}_j \mid \tilde{\xi}_j = \mathcal{T}(\xi_j, \xi_f^i),\ \xi_j \in o_i\}. \tag{9}$$

3.2.2. ECR-MADDPG

Unlike MADDPG that optimizes a set of actors and critics for all agents, ECR-MADDPG only learns a shared actor and a centralized critic. Thus, the policy (actor) and the critic are allowed to optimize with the experiences of all agents simultaneously [37]. Therefore, we store the experience of all agents in the form of a tuple $(s, \boldsymbol{o}, \boldsymbol{a}, \boldsymbol{r}, \boldsymbol{o}', s')$ into the experience replay buffer $\mathcal{D}$, where $\boldsymbol{o} = (o_1, \ldots, o_n)$, $\boldsymbol{a} = (a_1, \ldots, a_n)$, and $\boldsymbol{r} = (r_1, \ldots, r_n)$ represent the concatenation of all agent's observations, actions, and rewards, respectively. $\boldsymbol{o}' = (o_1', \ldots, o_n')$ and s' denote the observations of all agents and the global state after executing $\boldsymbol{a}$, respectively. By randomly sampling a minibatch from $\mathcal{D}$, we calculate the average gradient across all agents to update the shared policy. Derived from [36], the gradient can be written as

$$\begin{aligned} \nabla_{\theta^\mu} J(\theta^\mu) = \mathbb{E}_{(s,\boldsymbol{o},\boldsymbol{a})\sim\mathcal{D}}[\frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}}(\nabla_{\theta^\mu}\mu(\tilde{o}_i \mid \theta^\mu) \\ \times \nabla_{a_i} Q(\tilde{s}_i, a_1, \ldots, a_n \mid \theta^Q)\,|_{a_i=\mu(\tilde{o}_i|\theta^\mu)})], \end{aligned} \tag{10}$$

where μ parameterized by θ^μ represents the actor; Q parameterized by θ^Q denotes the associated critic.

Instead of concatenating the observations and actions of all agents as in MADDPG, we take the ego-centric system state $\tilde{s}_i$ and the joint actions $\boldsymbol{a}$ as the input to estimate the Q-values for each agent $i \in \mathcal{I}$. Formally, the critic parameters θ^Q are updated as

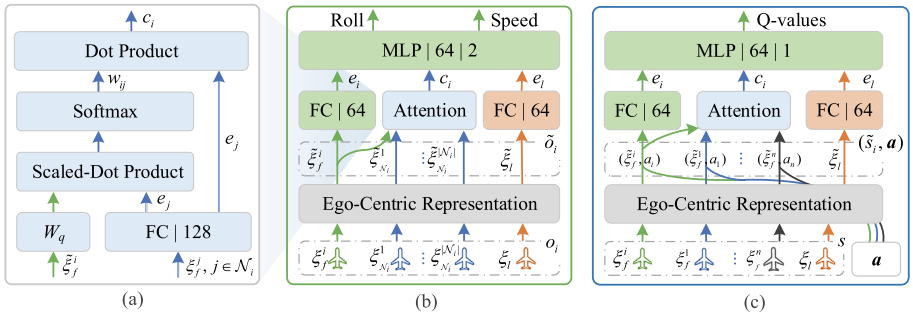


Fig. 3. The network structure of APINet. (a) Attention embedding module. (b) Actor network. (c) Critic network.

$$L(\theta^{Q}) = \mathbb{E}_{(s,\boldsymbol{o},\boldsymbol{a},\boldsymbol{r},s')\sim\mathcal{D}}[\frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}}(Q(\tilde{s}_i,\boldsymbol{a}\mid\theta^{Q}) - y_i)^2]. \tag{11}$$

The target Q-value y_i is defined as

$$y_i = r_i + \gamma Q^{-}(\tilde{s}_i', a_1', \ldots, a_n' \mid \theta^{Q^-})\,|_{a_j'=\mu^-\left(\tilde{o}_j|\theta^{\mu^-}\right)}, \tag{12}$$

where γ is the discounted factor; μ^- and Q^- denote the target networks for the actor and the critic, whose parameters θ^{μ^-} and θ^{Q^-} are updated as follows:

$$\begin{cases} \theta^{\mu^-} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu^-} \\ \theta^{Q^-} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q^-} \end{cases}, \tag{13}$$

where $\tau \ll 1$ is the soft update rate.

3.3. Knowledge transfer with APINet

In transfer learning, fine-tuning is a simple yet effective method to speed up the training process and improve sample efficiency. In this paper, fine-tuning refers to reloading the parameters of the actor and critic models learned in the previous stage, and using them as the initialization for the new models being trained in the current stage.

However, it is unfeasible to directly use this method to transfer knowledge in PASCAL. The main reason is that deep neural networks typically require a fixed-length input, while the state space in different learning stages is progressively increasing and the dimension of observation for each follower is time-varying. Consequently, there is a need for a customized network that can adapt to dynamic input dimensions and allow the reloading of the previously learned models. To this end, we leverage the self-attention mechanism [38] to develop a novel attention-based population-invariant network (APINet).

As depicted in Fig. 3, both the actor and the critic in ECR-MADDPG are implemented using APINet with a similar structure but different parameters. In the following, we first explain the actor network (Fig. 3(b)) and then the critic network (Fig. 3(c)).

3.3.1. Actor network

The action $a_i = \mu\left(\tilde{o}_i \mid \theta^{\mu}\right)$ is a function of agent i's ego-centric observation $\tilde{o}_i := (\tilde{\xi}_f^i, \tilde{\boldsymbol{\xi}}_{\mathcal{N}_i}, \tilde{\xi}_l)$. The dimension of the observation is uncertain because the number of neighbors varies with time. To solve this problem, we first use three embedding functions to separately encode the ego state, the neighbors' states, and the leader's state:

$$e_i = \mathrm{FC}(\tilde{\xi}_f^i), e_j = \mathrm{FC}(\tilde{\xi}_{\mathcal{N}_i}^j), e_l = \mathrm{FC}(\tilde{\xi}_l), \tag{14}$$

where e_i, e_j, and e_l are the embeddings of agent i's state, neighbor j's state, and the leader's state, respectively. FC$(\cdot)$ denotes the embedding function implemented by a one-layer fully-connected (FC) network activated by a rectified linear unit (ReLU).

Then, we design an attention embedding module (Fig. 3(a)) that aggregates the embeddings of all neighbors' states into a fixed-length content vector c_i:

$$c_i = \sum_{j\in\mathcal{N}_i} w_{ij} e_j. \tag{15}$$

Here the attention weight w_{ij} indicates the importance of the neighbor j to the ego agent i, which can be calculated as follows:

$$w_{ij} = \mathrm{Softmax}(\frac{[\tilde{\xi}_f^i]^{\mathrm{T}} W_q{}^{\mathrm{T}} e_j}{\sqrt{d_{e_j}}}), \tag{16}$$

where Softmax$(\cdot)$ is the softmax function, W_q is the parameter to learn, d_{e_j} is the dimension of e_j.

Finally, we determine the action by

$$a_i = \mathrm{MLP}(e_i \,||\, c_i \,||\, e_l), \tag{17}$$

where MLP$(\cdot)$ denotes a two-layer multi-layer perceptron (MLP) activated by a ReLU and a hyperbolic tangent (tanh) function; $||$ represents the concatenation operation.

3.3.2. Critic network

The critic network $Q(\cdot \mid \theta^{Q})$ has a similar structure as the actor network $\mu(\cdot \mid \theta^{\mu})$. The only difference between them is that the critic takes the ego-centric system state $\tilde{s}_i$ and the joint actions $\boldsymbol{a}$ as the input to estimate Q-values. The Q-value of agent i is computed as follows:

$$Q_i = Q(\tilde{s}_i, \boldsymbol{a} \mid \theta^{Q}) = \mathrm{MLP}(e_i \,||\, c_i \,||\, e_l), \tag{18}$$

where $e_i = \mathrm{FC}(\tilde{\xi}_f^i, a_i)$, $c_i = \sum_{j\in\mathcal{I}, j\neq i} w_{ij} e_j$, $e_j = \mathrm{FC}(\tilde{\xi}_f^j, a_j)$, $e_l = \mathrm{FC}(\tilde{\xi}_l)$; MLP$(\cdot)$ is a two-layer MLP network activated by a ReLU and a linear function.

By leveraging the designed APINet structure, each previously learned model can be easily reloaded as an initialization for the next learning stage, which dramatically improves the learning efficiency and final performance.

Finally, we summarize the proposed PASCAL approach in Algorithm 1.

4. Experiments and results

This section starts by introducing the parameter settings and evaluation metrics. Then, we conduct a series of numerical simulations to verify the effectiveness of our ECR-MADDPG algorithm, APINet structure, and PASCAL approach, respectively. Finally, we establish a semi-physical simulation system for further testing.

Algorithm 1 PASCAL with ECR-MADDPG.

Input: Maximum training episode per stage M; replay buffer maximum capacity N; maximum time step per episode N_s; training batch size N_b; update frequency of target networks C

Initialize a set of K learning stages
Initialize parameters θ_0^{μ} (for actor) and θ_0^{Q} (for critic) randomly
for stage $k = 1$ to K **do**
  Reload models from the previous stage: $\theta_k^{\mu} \leftarrow \theta_{k-1}^{\mu}$; $\theta_k^{Q} \leftarrow \theta_{k-1}^{Q}$; $\theta_k^{\mu^-} \leftarrow \theta_{k-1}^{\mu}$; $\theta_k^{Q^-} \leftarrow \theta_{k-1}^{Q}$ ▷ **Knowledge Transfer**
  Empty replay buffer $\mathcal{D}$ with capacity N
  Increase the number of followers n progressively
  for episode $e = 1$ to M **do** ▷ **Policy Learning**
    Initialize system state $s \leftarrow (\xi_l, \boldsymbol{\xi}_f)$ randomly
    Receive initial observations $\boldsymbol{o}$
    for $t = 1$ to N_s **do**
      Create ego-centric observation $\tilde{o}_i$ with (9) for each follower i
      Select action with current policy and Gaussian exploration noise: $a^i = \mu(\tilde{o}_i \mid \theta_k^{\mu}) + \mathbb{N}(0, \sigma^2)$
      Calculate ϕ_{ref}^i and v_{ref}^i with (1) and (2)
      $s' \leftarrow$ UAV Kinematics $(s, \boldsymbol{\phi}_{\text{ref}}, \boldsymbol{v}_{\text{ref}})$
      Calculate immediate reward r_i with (6)
      Receive new observations $\boldsymbol{o}'$
      Store $(s, \boldsymbol{o}, \boldsymbol{a}, \boldsymbol{r}, \boldsymbol{o}', s')$ in replay buffer $\mathcal{D}$, overwriting the oldest tuple if $|\mathcal{D}| > N$
      $s \leftarrow s'$; $\boldsymbol{o} \leftarrow \boldsymbol{o}'$
      Sample a minibatch of N_b tuples from $\mathcal{D}$ randomly
      Update θ_k^{μ} using the policy gradient (10)
      Update θ_k^{Q} through minimizing the loss (11)
      Update target network parameters $\theta_k^{\mu^-}$ and $\theta_k^{Q^-}$ with (13) every C time steps
    end for
  end for
end for

Table 1
Parameter Settings.

Name	Value	Name	Value
R_c	100 m	R_a	50 m
R_s	5 m	r_{bd}	$\frac{\pi}{6}$ rad
$v_{\max}$	18 m/s	$v_{\min}$	12 m/s
$a_{\max}^r$	$\frac{\pi}{18}$ rad	$a_{\max}^v$	1 m/s
σ	$1.0 \rightarrow 0.05$	P	1000
N_b	32	N_s	60
N	100000	γ	0.95
w_1	5	w_2	10
τ	0.01	C	10

4.1. Experimental setup

4.1.1. Parameter settings

We used the PyTorch library in Python to implement our algorithm. The embedding functions (i.e., FC(·) in Fig. 3) for the ego agent, the leader, and the neighbors had 64, 64, and 128 neurons, respectively. The parameter $W_q \in \mathbb{R}^{128\times 5}$. For the critic, the two-layer MLP had 64 and 1 neuron(s), while the MLP(·) for the actor consisted of two hidden layers with 64 and 2 neurons. All network parameters are optimized using the Adam optimizer with learning rates 0.001 (for actor) and 0.0001 (for critic). We trained with five random seeds for all experiments.

The proposed algorithm was trained for a total of 10000 episodes. During training, the Gaussian exploration parameter σ was annealed exponentially from the initial value 1.0 to the minimum value 0.05. In each episode, the system state is initialized randomly. At each time step, the leader's behavior was determined randomly. Table 1 provides the empirical values for the parameter settings.

4.1.2. Performance metrics

In order to evaluate the effectiveness and efficiency of different algorithms quantitatively, we elaborated the performance metrics as follows.

- **Average reward** (G): We defined the average reward over a certain number of episodes (e.g., N_e episodes) to evaluate the policy performance during training and testing:

$$G = \frac{1}{nN_eN_s}\sum_{i=1}^{n}\sum_{e=1}^{N_e}\sum_{t=1}^{N_s} r_{i,t}^e, \tag{19}$$

where $r_{i,t}^e$ was the reward obtained by the follower i at the time step t of the episode e.

- **Collision rate** (F): We considered that a collision would occur if the distance between two given UAVs was less than the minimum safety distance R_s. Accordingly, the collision rate was defined as

$$F = \frac{1}{nN_eN_s}\sum_{i=1}^{n}\sum_{e=1}^{N_e}\sum_{t=1}^{N_s} \mathbb{1}(\rho_i < R_s \text{ or } d_i < R_s), \tag{20}$$

where $d_i = \min\{D_{ij} \mid j \in \mathcal{N}_i\}$, $\mathbb{1}(\cdot)$ was the indicator function.

- **Average distance** ($\bar{\rho}$): The average distance between the leader and the followers was calculated as

$$\bar{\rho} = \frac{1}{nN_eN_s}\sum_{i=1}^{n}\sum_{e=1}^{N_e}\sum_{t=1}^{N_s} \rho_i. \tag{21}$$

- **Average heading difference** ($\Delta\bar{\psi}$): The average heading difference between the leader and the followers was defined as

$$\Delta\bar{\psi} = \frac{1}{nN_eN_s}\sum_{i=1}^{n}\sum_{e=1}^{N_e}\sum_{t=1}^{N_s} \Delta\psi_i. \tag{22}$$

4.2. Simulation results for ECR-MADDPG

We first compared our ECR-MADDPG algorithm with the vanilla MADDPG algorithm [36]. In the experiment, we only considered the two closest neighbors because MADDPG was unable to handle the dynamic inputs. After concatenating the ego state, the leader state, and the neighbors' state, the actor of MADDPG directly mapped this concatenation to the action using an MLP network. This MLP consisted of three hidden layers with 256, 64, and 2 neurons, followed by two ReLU and one tanh activation function, respectively. The critic of MADDDPG had a similar structure with its actor, except that the last layer of the MLP had one neuron and the linear activation function. To make a fair comparison, ECR-MADDPG used the same network structure and parameter settings as MADDPG. We used the two algorithms to learn the flocking policy for the UAV swarms with five followers and ten followers, respectively.

We report the training results of the two algorithms for each swarm scale in Fig. 4. In the scenario with $n = 5$ followers, the learning curve of ECR-MADDPG grows at a similar speed as MADDPG in the early phase. Both algorithms converge at about 4000 episodes. However, the final reward obtained by ECR-MADDPG is approximately 27% higher than that of MADDPG. In the scenario with $n = 10$ followers, the average reward received by MADDPG increases slowly throughout the learning process, whereas that of ECR-MADDPG increases rapidly in the early phase and gradually stabilizes after 6000 episodes. Finally, ECR-MADDPG receives roughly 56% higher reward than MADDPG. The main reason is that the parameter sharing with ego-centric representation mechanism dramatically reduces the number of parameters and improves the learning efficiency. The above results demonstrate that the proposed ECR-MADDPG algorithm outperforms the vanilla MADDPG algorithm in both learning efficiency and final performance, especially when the swarm scale is large.

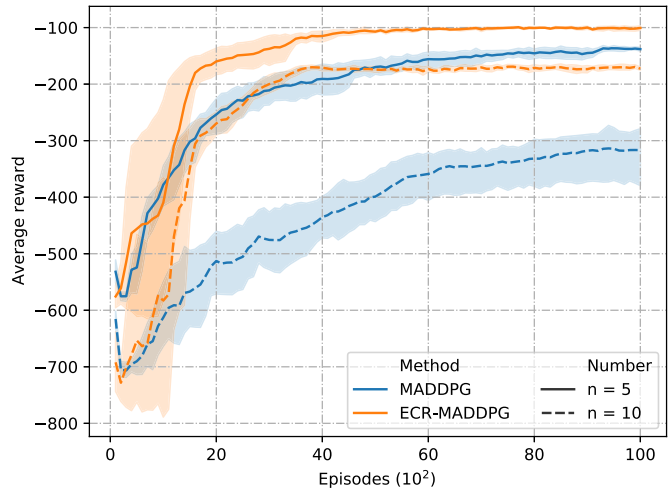


Fig. 4. Learning curves of MADDPG and ECR-MADDPG for UAV swarms with different numbers of followers. Note that the average reward was recorded every $N_e = 100$ episodes.

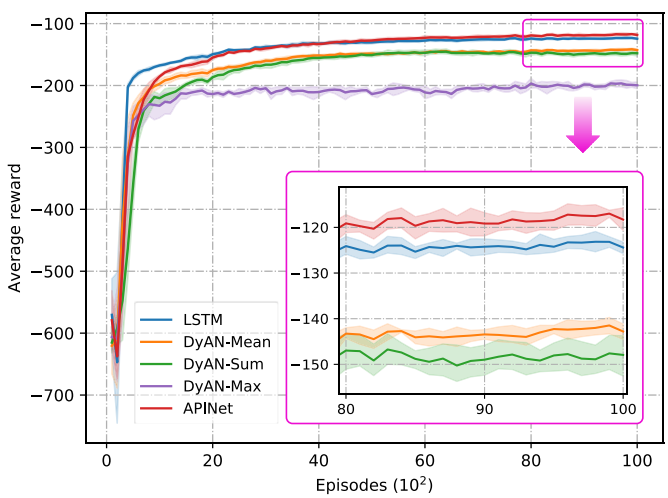


Fig. 5. Learning curves of ECR-MADDPG with different network structures. Note that the average reward was recorded every $N_e = 100$ episodes.

4.3. Simulation results for APINet

We then investigated the performance of the designed APINet structure by comparing it with several benchmarks.

4.3.1. Training results

We compared our APINet with two baselines, i.e., LSTM [39] and the dynamic agent-number network (DyAN) [33]. Specifically, the attention embedding model in APINet (Fig. 3(a)) was replaced with LSTM or the three aggregation operators in DyAN, i.e., the MAX operator, the MEAN operator, and the SUM operator. The number of LSTM cells was set to 128, and the rest of the network structure remained the same as APINet. The above five embedding methods were trained using the ECR-MADDPG algorithm with the same parameter settings.

Fig. 5 records the learning curves of different methods when the swarm scale $n = 10$. Intuitively, DyAN-Max has the worst performance. DyAN-Sum and DyAN-Mean obtain similar performance in learning efficiency and final reward. Compared with DyAN, LSTM and APINet receive higher average rewards. The main reason is that LSTM and APINet distinguish the importance of different neighbors, while DyAN treats all neighbors equally. By comparing the learning curves of LSTM and APINet, we can find that the average reward of LSTM grows faster than APINet in the very early phase. Whereas, with the increase of training episodes, the average reward of APINet gradually exceeds that of LSTM after about 4000 episodes. A reasonable explanation is that LSTM uses priori knowledge to distinguish the importance of different neighbors, i.e., the closer neighbors have more influence on the ego follower. Instead, APINet learns to distinguish such importance through the attention mechanism. As the training episode increases, the importance learned by APINet is more accurate than the priori knowledge. The above results verify that our APINet structure outperforms the four existing state-of-the-art network structures.

Table 2
Quantitative Testing Results of ECR-MADDPG with Different Network Structures.

Structure	G	F (‰)	$\bar{\rho}$ (m)	$\Delta\bar{\psi}$ (°)
LSTM [39]	-105.07±3.70	0.56±0.15	**68.27±1.09**	**10.29 ±0.78**
DyAN-Mean [33]	-128.67±3.80	1.57 ±0.64	95.33±1.93	17.17±1.39
DyAN-Sum [33]	-140.49±8.24	1.57±0.20	105.24±10.89	19.38±2.04
DyAN-Max [33]	-182.26±8.92	6.63±1.73	121.89±13.13	22.33±2.27
APINet (Ours)	**-95.39±2.73**	**0.47±0.09**	75.61±3.66	11.90±1.09

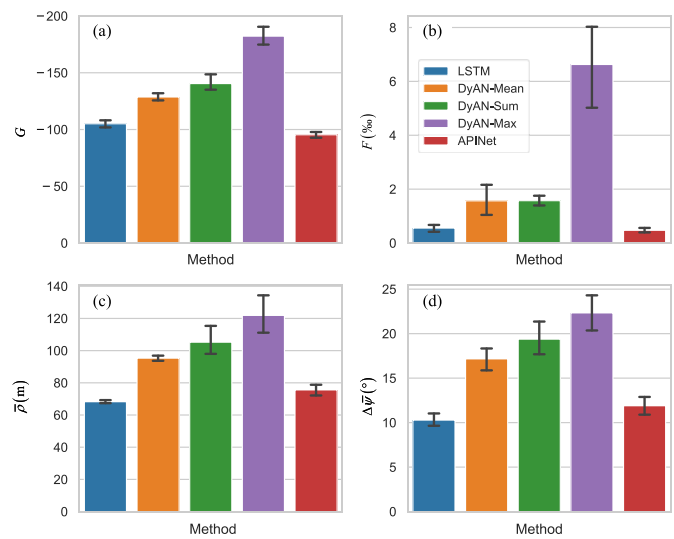


Fig. 6. Performance of ECR-MADDPG with different network structures. The error bar is used to show the standard error. Note that the higher the bar, the worse the performance. (a) Average reward (Note that the vertical axis is inversed). (b) Collision rate. (c) Average distance. (d) Average heading difference.

4.3.2. Testing results

After training, we tested the policies learned with the five network structures in the scenario with $n = 10$ followers. Specifically, we tested with five random seeds for each policy. Each testing consisted of $N_e = 20$ episodes, and each episode lasted for $N_s = 180$ time steps.

Table 2 and Fig. 6 compare the performance of the five policies. As can be seen, compared with APINet and LSTM, the performance of the three DyAN-based structures is much worse. This means that distinguishing the importance of different neighbors helps to improve the algorithm performance. In addition, APINet and LSTM achieve very similar performance. Although the average distance and the average heading difference of APINet are lower than that of LSTM, the collision rate and the average reward are much better. This result shows that the collision-free flocking policy learned with our APINet structure has higher safety and overall performance.

Table 3
Training time of PASCAL, multi-stage learning from scratch, and learning from scratch.

Approach	From scratch	From scratch (multi-stage)	PASCAL
Time (h)	12.15	5.39	5.68

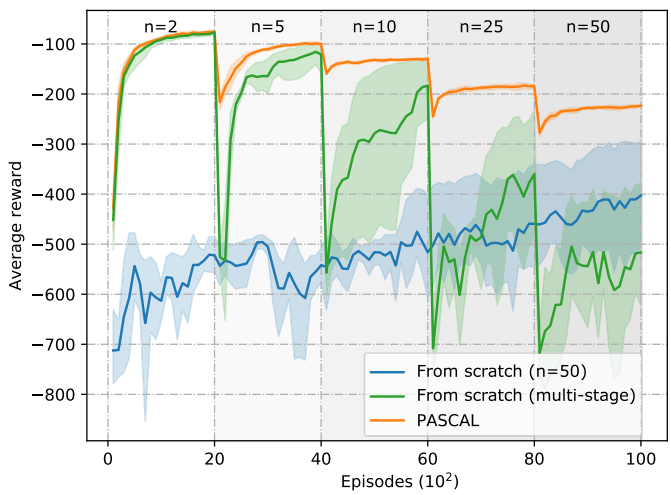


Fig. 7. Learning curves of PASCAL, multi-stage learning from scratch, and learning from scratch. Note that the average reward was recorded every $N_e = 100$ episodes (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.).

4.4. Simulation results for PASCAL

We further conducted ablation experiments to verify the effectiveness of the proposed PASCAL approach.

4.4.1. Training results

In this experiment, we designed a curriculum containing five learning stages for PASCAL, each of which learned on a UAV swarm with different number of followers. Specifically, we started to train a collision-free flocking policy with 2 followers and a leader from scratch. Upon convergence, we increased the number of followers to 5, and further optimized the policy that transferred from the previous stage. After that, we reloaded the policy through the knowledge transfer procedure, and continued the policy learning procedure with 10 followers. In the next two stages, the two procedures continued for 25 followers and 50 followers, respectively. For comparison, we trained a collision-free flocking policy in each stage without reloading the previous model. In addition, instead of dividing the learning process into five stages and progressively increasing the number of followers, we also directly trained a collision-free flocking policy on a UAV swarm with 50 followers from scratch. We term the two learning manners as *multi-stage learning from scratch* and *learning from scratch*.

We used the above three approaches (i.e., PASCAL, multi-stage learning from scratch, and learning from scratch) to learn the collision-free flocking policies separately. The training times are reported in Table 3. Although the consumption of multi-stage learning from scratch is similar to that of PASCAL, it fails to learn a convergent flocking policy (see Fig. 7). We note that our PASCAL takes only 5.68 hours to obtain a convergent flocking policy, which is significantly faster than learning from scratch. This result demonstrates the advantage of our proposed PASCAL approach in terms of training costs.

Fig. 7 depicts the average rewards over each $N_e = 100$ episodes of the three approaches. As shown, when the swarm scale is small, e.g., $n = 2$ or $n = 5$, multi-stage learning from scratch is able to converge. With the number of followers increasing from 10 to 50, convergence becomes more and more difficult (green line in Fig. 7). When $n = 50$, even after 10000 episodes of training, the average reward obtained by learning from scratch only reaches about -400 with a high variance (blue line in Fig. 7). In contrast, PASCAL can always converge to a better reward with a smaller variance within a shorter time, insensitive to the scale of the UAV swarm (orange line in Fig. 7). In other words, both the learning efficiency and the final performance are significantly improved by PASCAL, particularly for large-scale UAV swarms. The comparison results demonstrate the high efficiency and strong stability of the proposed PASCAL approach.

4.4.2. Testing results

After training, we tested the collision-free flocking policy learned by PASCAL (hereafter referred to as the learned PASCAL policy for short) with a typical task. This testing lasted for 180 time steps, i.e., 180 seconds. Initially, the leader was followed by 50 followers. The initial position of the followers was located in a square area of size 500 m$\times$ 500 m (black dotted square in Fig. 8(a)). We divided this area into 10 $\times$ 5 grids, and each follower was located in the center of the grid. After 60 time steps, the other 25 followers joined the swarm simultaneously. At that moment, the 25 followers were located in the centers of the grids with size of 50 m$\times$ 50 m (black dotted square in Fig. 8(b)). At the time step 120, the 25 followers quit the swarm. During this testing, the leader selected its velocity setpoint from the interval $(12, 18)$ randomly. Meanwhile, its roll angle setpoint was randomly generated from the intervals $(-\frac{\pi}{9}, -\frac{\pi}{12})$ (during 30 $\sim$ 65 s), $(\frac{\pi}{36}, \frac{\pi}{12})$ (during 100 $\sim$ 160 s), or $(-\frac{\pi}{36}, \frac{\pi}{36})$ (otherwise).

The trajectories of the UAV swarm are visualized in Fig. 8. The red line denotes the trajectory of the leader, while the blue line represents the trajectory of the virtual follower, i.e., the center of all the followers. We plot the trajectories of all the followers at different time intervals in Fig. 8(a)-(d), respectively. As shown, the trajectories of the leader and the virtual follower almost overlap, even if the flight direction of the leader and the number of followers are changed dramatically. This result confirms that the learned PASCAL policy can adapt to dynamic large-scale UAV swarms with an uncertain number of followers. Using this policy, all the followers are relatively evenly distributed around the leader and are able to follow the leader successfully.

To better analyze the performance of the learned PASCAL policy, we record the distances between the leader and the followers (ρ), the differences in heading between the leader and the followers ($\Delta\psi$), as well as the minimum distance between the followers (*mindis*) in Fig. 9(a)-(c) with respect to time, respectively. It can be observed that the differences in heading between the leader and the followers are kept between $-30°$ to $30°$ most of the time, which means the followers can maintain a similar flight direction to the leader. It should be noted that the minimum value of *mindis* and the minimum value of ρ are both around 10 m (the accurate values are 10.93 m and 8.53 m), which is large than the minimum safety distance $R_s = 5$ m. This indicates there is no collision between the follower and the leader throughout this testing experiment. The above results demonstrate that using the learned PASCAL policy, the followers are able to flock with the leader and avoid collisions between each other at the same time.

4.4.3. Generalization results

In addition to the above testing experiments, we also evaluated the generalization ability of the learned PASCAL policy. To do so, we took the best policy trained by PASCAL on the UAV swarm with $n = 50$ followers and directly applied this policy to a new UAV swarm with different number of followers. We evaluated the policy on the scenario with 10 $\sim$ 100 followers, respectively. We

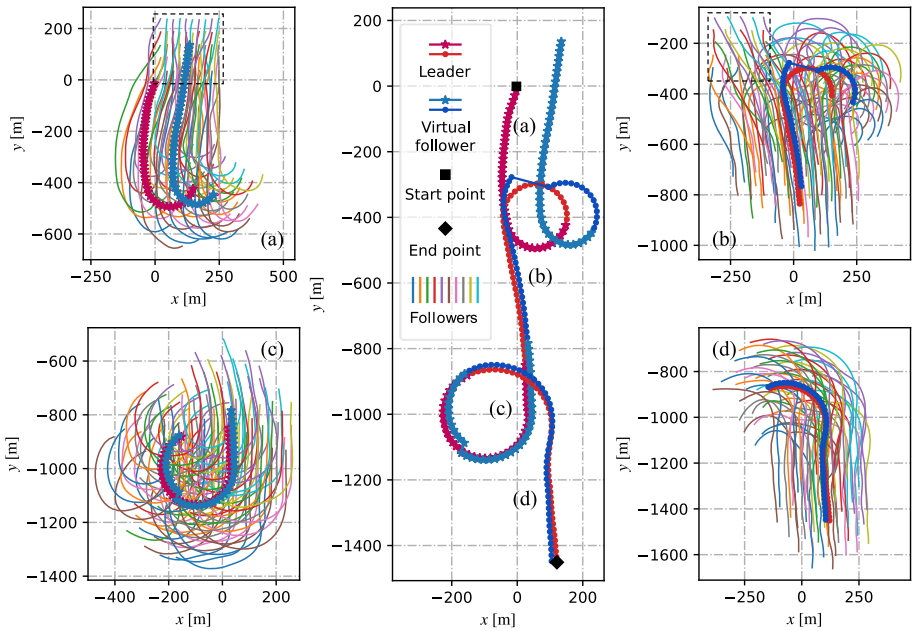


Fig. 8. Trajectories of 75 followers using the learned PASCAL policy to flock with the leader. Note that the virtual follower denotes the center of all followers. (a) $t = 0 \sim 45$ s. (b) $t = 45 \sim 90$ s. (c) $t = 90 \sim 135$ s. (d) $t = 135 \sim 180$ s.

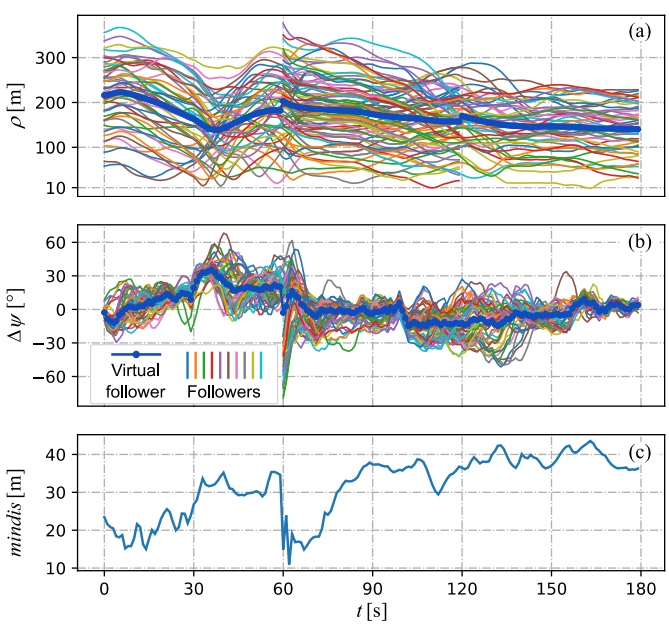


Fig. 9. Performance of the learned PASCAL policy. (a) The distances between the leader and the followers. (b) The differences in heading between the leader and the followers. (c) The minimum distance between the followers.

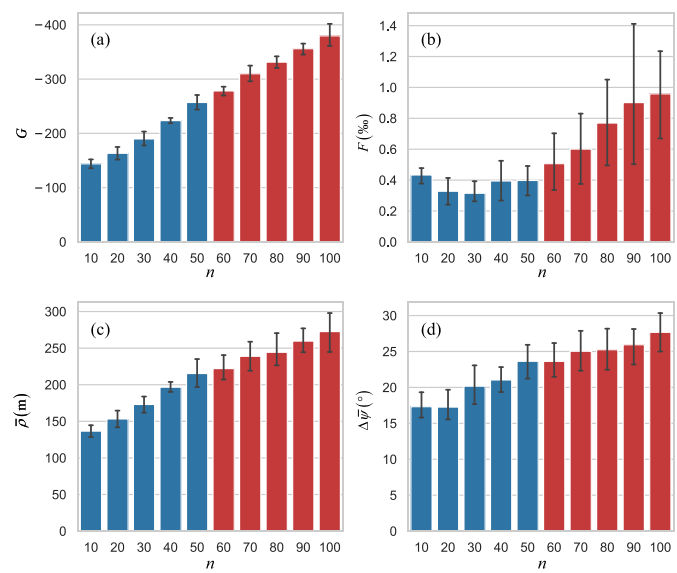


Fig. 10. Generalization performance of the learned PASCAL policy for UAV swarms with different number of followers. The red bars mean a larger swarm scale than the training ones, and the black error bar is used to show the standard error. Note that the higher the bar, the worse the performance. (a) Average reward (Note that the vertical axis is inversed). (b) Collision rate. (c) Average distance. (d) Average heading difference.

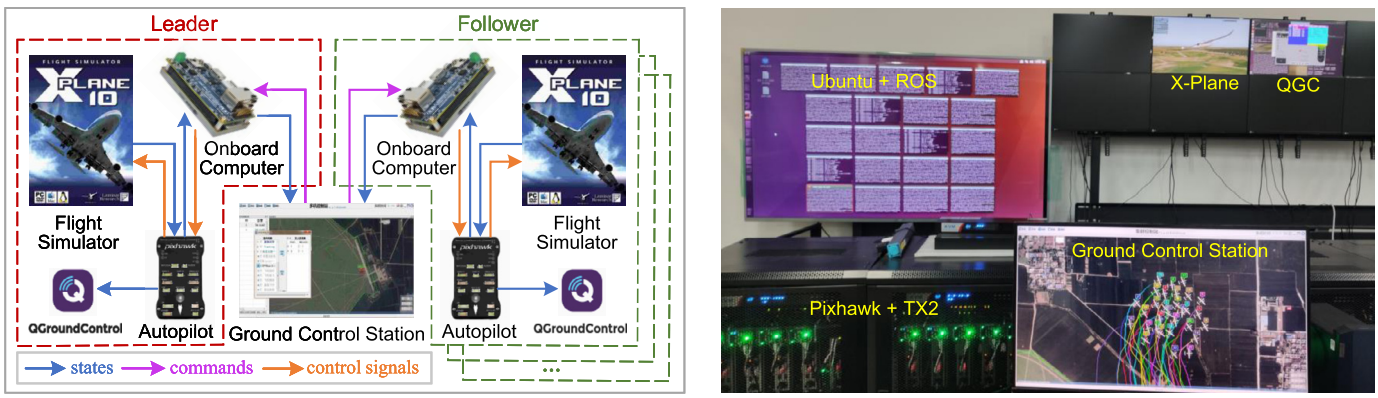
evaluated with 5 random seeds for each swarm scale. Each evaluation consisted of $N_e = 20$ episodes, and each episode lasted for $N_s = 180$ time steps.

Table 4 and Fig. 10 present the zero-shot generalization results. It is intuitive that the performance in terms of the four metrics is better in the case of smaller swarm scales than in the case of larger swarm scales. Specifically, as the population of UAVs in the swarm increases, the collision rate, average distance, and average heading difference gradually increase, resulting in a gradual decrease in average reward. This is because the increase in the number of followers inevitably leads to an increase in the distance between the leader and the further followers, as well as the risk of collisions between the leader and the closer followers. It should be noted that the collision rate is always less than 1‰, even if the number of followers increases by 100% or reduces by 80%. This means the performance of PASCAL is satisfying, irrelevant with the swarm scale. The above results demonstrate that the learned PASCAL policy can be generalized to different swarm scales without any fine-tuning.

We also report the average inference time of each decision-making step (i.e., T_{inf}) in Table 4. When the size of the swarm is small, the average inference time roughly increases with the number of followers. With the further increase in the number of followers, the average inference time gradually levels off. Even with 100 followers, each follower only spends less than 1.2 ms to determine the action using the learned PASCAL policy. This result means

Table 4
Generalization Testing Results of PASCAL for the UAV Swarm with Different Number of Followers.

n	G	F (‰)	$\bar{\rho}$ (m)	$\Delta\bar{\psi}$ (°)	T_{inf} (ms)
10	-143.88 ± 9.67	0.43 ± 0.05	136.61 ± 9.28	17.32 ± 2.10	0.71
20	-163.28 ± 13.80	0.33 ± 0.10	153.26 ± 13.46	17.26 ± 2.44	0.85
30	-189.78 ± 13.13	0.31 ± 0.08	173.02 ± 13.65	20.17 ± 3.20	1.08
40	-223.54 ± 5.55	0.39 ± 0.15	196.60 ± 7.67	21.04 ± 2.08	1.11
50	-256.97 ± 16.37	0.40 ± 0.11	215.32 ± 22.86	23.63 ± 2.84	1.08
60	-278.22 ± 9.45	0.51 ± 0.21	222.05 ± 19.22	23.63 ± 2.86	1.07
70	-309.85 ± 16.42	0.60 ± 0.27	238.82 ± 24.09	25.03 ± 3.21	1.11
80	-331.24 ± 12.42	0.77 ± 0.32	244.29 ± 26.59	25.25 ± 3.26	1.11
90	-356.05 ± 11.16	0.90 ± 0.52	259.60 ± 20.40	25.94 ± 2.77	1.15
100	-379.70 ± 24.73	0.96 ± 0.33	272.60 ± 30.11	27.67 ± 2.97	1.10



(a) (b)

Fig. 11. The high-fidelity semi-physical simulation system. (a) Key components. (b) Experimental platform.

that the proposed PASCAL approach can achieve flocking control for large-scale UAV swarms with good real-time performance.

4.5. Semi-physical simulation results

We finally tested the feasibility of the learned PASCAL policy by conducting the hardware-in-loop (HIL) experiments in a high-fidelity semi-physical simulation system.

4.5.1. Semi-physical simulation system

We extended our high-fidelity semi-physical simulation system [28,40] for multiple fixed-wing UAV scenarios. As depicted in Fig. 11, each UAV was composed of an X-Plane 10 flight simulator, a PX4 / Pixhawk autopilot, and an NVIDIA Jetson TX2 onboard computer. All UAVs shared one ground control station. All onboard computers were accessed to a local area network for simulating the wireless communication channel between UAVs. We ran the Ubuntu 16.04 LTS and Robot Operating System (ROS) on each onboard computer. The learned PASCAL policy was running independently on this platform.

4.5.2. Simulation results

In this HIL experiment, 29 followers (UAV2 $\sim$ 30 in Fig. 12) employed the learned PASCAL policy to follow the leader (UAV1 in Fig. 12). This experiment lasted for 240 time steps, i.e., 4 minutes. At each time step, the leader selected its roll angle and velocity setpoints according to the pre-planned paths, and broadcasted its state information to the followers via the UDP socket. Based on the leader's state, the neighbors' states, and the ego state, each follower selected its roll and velocity actions with the learned PASCAL policy. Lastly, the roll angle and velocity setpoints were determined and then published to the autopilot to carry out the closed-loop control with a PID controller.

The trajectory results of this HIL experiment are depicted in Fig. 12. As can be seen, the followers are always able to follow the leader steadily, even if the leader changes its heading sharply. In addition, the trajectory of the virtual follower is very similar to that of the leader, which indicates the similar behavior between them. Even after the 4-minute flight test, the followers still follow the leader with a relatively good performance.

We also record the distances between the leader and the followers (ρ), the differences in heading between the leader and the followers ($\Delta\psi$), as well as the minimum distance between the followers (*mindis*) in Fig. 13. As shown, during this flight, the average distance between the leader and all the followers is maintained between 100 and 200 meters most of the time. Besides, the differences in heading between the leader and the followers are kept between $-45°$ to $45°$ most of the time. The results mean that all the followers are able to keep up with the leader. More importantly, the minimum distance between the followers is maintained around 30 m (the average value is 27.88 m), the minimum value of *mindis* is larger than the minimum safety distance $R_s = 5$ m (the accurate value is 5.64 m, when $t = 181$). This result shows that the learned PASCAL policy enables the followers to avoid conflicts with each other.

We noted that the PASCAL policy employed by the 29 followers in the HIL experiment is learned from the previous numerical simulation environment with $n = 25$ followers, without any parameter fine-tuning. This demonstrates that the learned PASCAL policy can not only adapt to different swarm scale, but also can be directly generalized to new situations.

4.5.3. Keyframe analysis

In addition to the global trajectories and the above performance metrics, we then explored the interpretability of the learned PASCAL policy. To this end, we choose an interesting moment in the HIL experiment, and visualize the attention weights of all followers

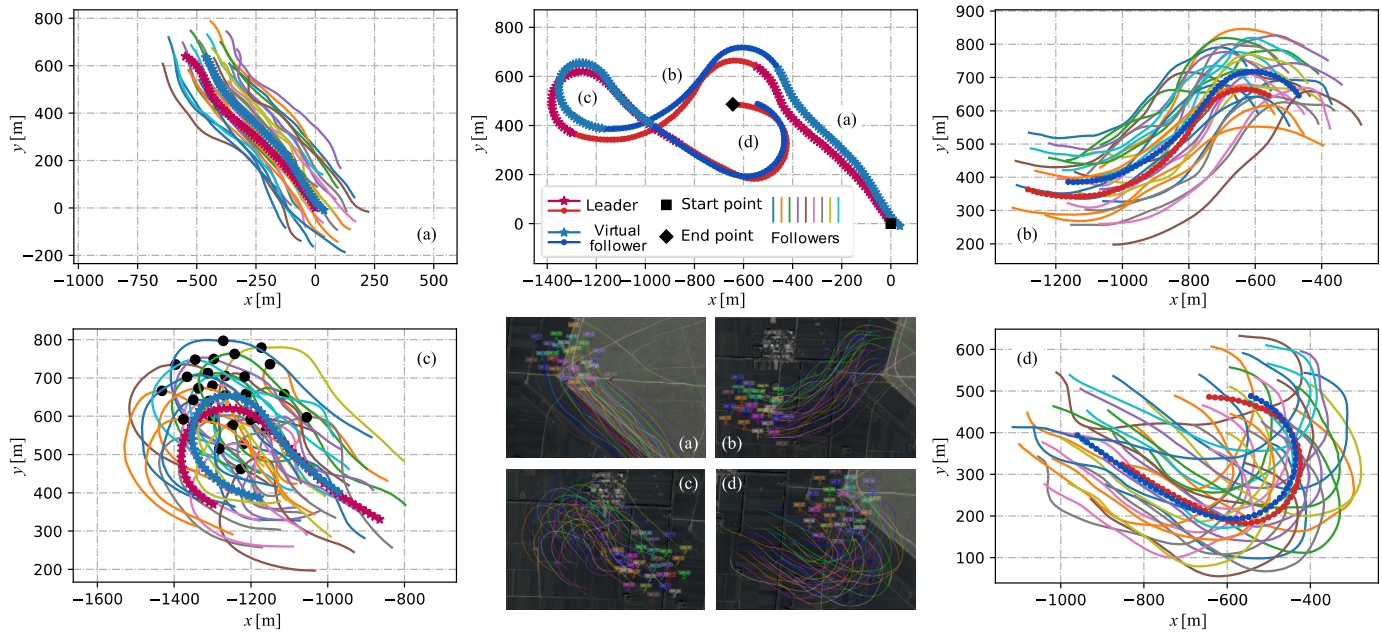


Fig. 12. Trajectories of 29 followers flock with the leader via the learned PASCAL policy in semi-physical simulation. The four representative snapshots displayed on the bottom-middle are captured from the ground control station. Note that the virtual follower denotes the center of all followers. (a) $t = 0 \sim 60$ s. (b) $t = 60 \sim 120$ s. (c) $t = 120 \sim 180$ s. (d) $t = 180 \sim 240$ s. A video about the semi-physical simulation is available at https://youtu.be/LeouTlP4090.

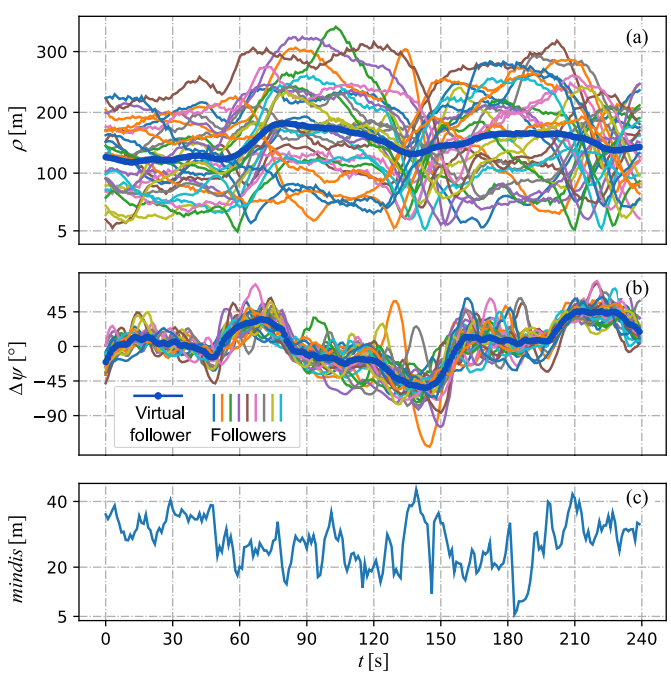


Fig. 13. Performance of the learned PASCAL policy in the semi-physical simulation. (a) The distances between the leader and the followers. (b) The differences in heading between the leader and the followers. (c) The minimum distance between the followers.

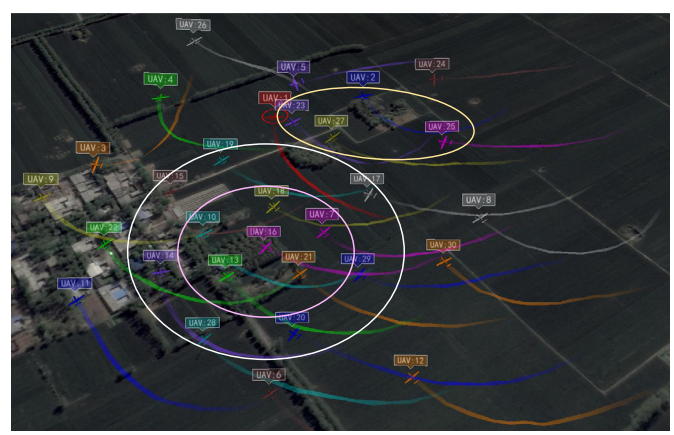


Fig. 14. The keyframe snapshot of the trajectories in third-person view at $t = 150$ s.

for analysis. At this moment, i.e., $t = 150$ s, the leader just finished a sharp turn and was resuming to a relatively straight movement, as shown by the black dots in Fig. 12(a). The keyframe snapshot of the trajectories is presented in Fig. 14, and the corresponding attention weights of all the followers are visualized in Fig. 15.

In Fig. 15, the vertical axis denotes each follower's ID in the semi-physical simulation system, and the horizontal axis represents the neighbor's ID corresponding with each follower. As can be seen, the followers pay more attention to their closer neighbors. To figure this out, we take UAV16 as an example for analysis. At the moment $t = 150$ s, UAV16 had 12 neighbors (white ellipse in Fig. 14), among which UAV7, UAV10, UAV13, UAV18, UAV21 (purple ellipse in Fig. 14) were closer than the other 7 neighbors. As a result, UAV16 paid more attention to the 5 closer neighbors, while the other 7 neighbors only had almost 0.00 attention, as shown in Fig. 15. However, this does not mean that the closer the distance, the greater the attention weight. The distances between the follower (UAV16) and the 5 closer neighbors were 42.06 (UAV7), 42.02 (UAV10), 34.78 (UAV13), 41.14 (UAV18), 37.19 (UAV21) meters, respectively, while the corresponding attention weights were 0.02, 0.13, 0.54, 0.21, 0.09. Although the distances from UAV16 to UAV7, UAV10, and UAV18 were almost the same, the corresponding attention weights were significantly different. The explanation is that other factors, such as the relative heading, roll, and speed, also influence the distribution of the attention weights. UAV27 is another example for this explanation. As shown in the yellow ellipse in Fig. 14, UAV25 was closing fast on UAV27, while UAV23 was moving away from UAV27. In this case, UAV27 should have paid more attention to UAV25, because UAV25 brought a greater

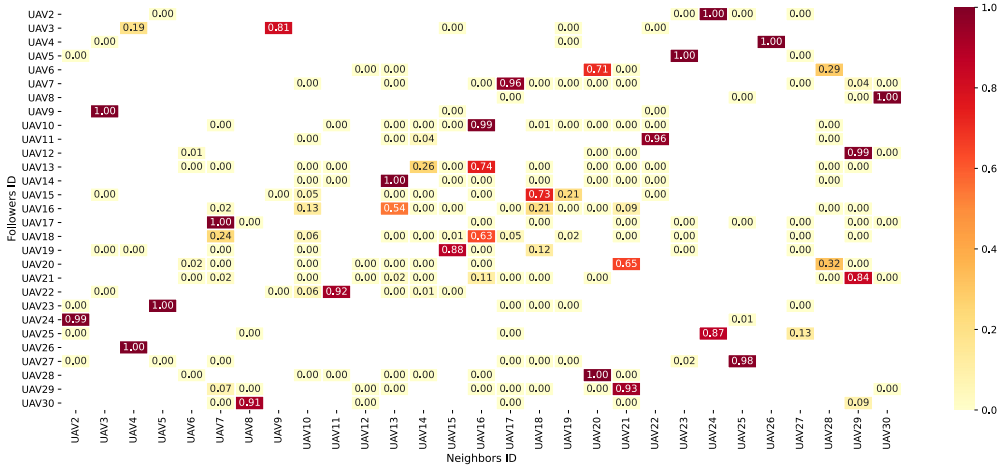


Fig. 15. Heatmap of the attention weights for all the followers at $t = 150$ s.

risk of collision to UAV27. As expected, UAV27 had 0.98 attention to UAV25, while other neighbors only had 0.02 attention at most, including the closest neighbor UAV23. The above results demonstrate that the learned PASCAL policy with the APINet structure is able to identify valuable neighbors automatically.

5. Conclusion

In this paper, we have solved the collision-free flocking control problem for scalable and large-scale fixed-wing UAV swarms by integrating curriculum learning into multi-agent deep reinforcement learning. In contrast to previous work, this problem is more challenging due to the consideration of collision avoidance as well as the large population of UAVs in the swarm. The key insight behind the proposed PASCAL approach is to divide the training process into multiple learning stages and progressively expand the scale of UAV swarms. The numerical and semi-physical simulations have verified the remarkable advantages of PASCAL in learning efficiency, generalization, and interpretability.

There are still some issues worthy of further investigation. Firstly, the population-specific curriculum in our work is handcrafted with expert experience. The generated curricula may be too easy or too difficult for the agents, which may lead to inefficiency or non-convergence. A meaningful future direction lies in achieving automatic curriculum learning to improve the learning efficiency and performance for large-scale MADRL problems. Secondly, this work only focuses on the flocking problem for fixed-wing UAVs in a two-dimensional plane. In the future, we will further evaluate the proposed approach under different parameter configurations and develop the MADRL-based method for flocking with fixed-wing UAVs in three-dimensional environments.

CRediT authorship contribution statement

Chao Yan: Investigation, Methodology, Software, Writing – original draft. **Xiaojia Xiang:** Resources, Supervision. **Chang Wang:** Formal analysis, Writing – review & editing. **Fei Li:** Conceptualization. **Xiangke Wang:** Resources, Writing – review & editing. **Xin Xu:** Writing – review & editing. **Lincheng Shen:** Project administration, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] M. Zhang, H. Wang, J. Wu, On uav source seeking with complex dynamic characteristics and multiple constraints: a cooperative standoff monitoring mode, Aerosp. Sci. Technol. 121 (2022) 107315.
[2] W. Dai, B. Pang, K.H. Low, Conflict-free four-dimensional path planning for urban air mobility considering airspace occupancy, Aerosp. Sci. Technol. 119 (2021) 107154.
[3] X. Zhang, H. Zhao, J. Wei, C. Yan, J. Xiong, X. Liu, Cooperative trajectory design of multiple uav base stations with heterogeneous graph neural networks, IEEE Trans. Wirel. Commun. (2022).
[4] Z. Liu, X. Wang, L. Shen, S. Zhao, Y. Cong, J. Li, D. Yin, S. Jia, X. Xiang, Mission-oriented miniature fixed-wing uav swarms: a multilayered and distributed architecture, IEEE Trans. Syst. Man Cybern. Syst. 52 (3) (2022) 1588–1602.
[5] J. Hu, H. Niu, J. Carrasco, B. Lennox, F. Arvin, Fault-tolerant cooperative navigation of networked uav swarms for forest fire monitoring, Aerosp. Sci. Technol. 123 (2022) 107494.
[6] Y. Shen, C. Wei, Multi-uav flocking control with individual properties inspired by bird behavior, Aerosp. Sci. Technol. (2022) 107882.
[7] Z. Zhou, H. Xu, Large-scale multiagent system tracking control using mean field games, IEEE Trans. Neural Netw. Learn. Syst. 33 (10) (2022) 5602–5610.
[8] A. Khan, E. Tolstaya, A. Ribeiro, V. Kumar, Graph policy gradients for large scale robot control, in: Conference on Robot Learning, PMLR, 2020, pp. 823–834.
[9] L.E. Beaver, A.A. Malikopoulos, An overview on optimal flocking, Annu. Rev. Control 51 (2021) 88–99.
[10] P. Zhu, W. Dai, W. Yao, J. Ma, Z. Zeng, H. Lu, Multi-robot flocking control based on deep reinforcement learning, IEEE Access 8 (2020) 150397–150406.
[11] F. Wang, J. Huang, K.H. Low, Z. Nie, T. Hu, Agds: adaptive goal-directed strategy for swarm drones flying through unknown environments, Complex Intell. Syst. (2022) 1–16.
[12] O. Saif, I. Fantoni, A. Zavala-Río, Distributed integral control of multiple uavs: precise flocking and navigation, IET Control Theory Appl. 13 (13) (2019) 2008–2017.
[13] W. Zhao, H. Chu, M. Zhang, T. Sun, L. Guo, Flocking control of fixed-wing uavs with cooperative obstacle avoidance capability, IEEE Access 7 (2019) 17798–17808.
[14] Y. Lyu, J. Hu, B.M. Chen, C. Zhao, Q. Pan, Multivehicle flocking with collision avoidance via distributed model predictive control, IEEE Trans. Cybern. 51 (5) (2021) 2651–2662.
[15] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, Q. Miao, Deep reinforcement learning: a survey, IEEE Trans. Neural Netw. Learn. Syst. (2022).
[16] C. Yan, X. Xiang, C. Wang, Towards real-time path planning through deep reinforcement learning for a uav in dynamic environments, J. Intell. Robot. Syst. 98 (2020) 297–309.
[17] Coordinated Intelligent Control of the Flight Control System and Shape Change of Variable Sweep Morphing Aircraft Based on Dueling-Dqn, Aerosp. Sci. Technol. (2022) 107898.
[18] H. Yuan, D. Li, Deep reinforcement learning for rendezvous guidance with enhanced angles-only observability, Aerosp. Sci. Technol. 129 (2022) 107812.
[19] C. Wang, J. Wang, X. Zhang, A deep reinforcement learning approach to flocking and navigation of uavs in large-scale complex environments, in: 2018 IEEE

Global Conference on Signal and Information Processing (GlobalSIP), Anaheim, CA, USA, 2018, pp. 1228–1232.
[20] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, in: International Conference on Learning Representations (ICLR), 2016.
[21] P. Yan, C. Bai, H. Zheng, J. Guo, Flocking control of uav swarms with deep reinforcement learning approach, in: 2020 3rd International Conference on Unmanned Systems (ICUS), IEEE, 2020, pp. 592–599.
[22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint, arXiv:1707.06347, 2017.
[23] Y. Zhi, L. Liu, B. Guan, B. Wang, Z. Cheng, H. Fan, Distributed robust adaptive formation control of fixed-wing uavs with unknown uncertainties and disturbances, Aerosp. Sci. Technol. 126 (2022) 107600.
[24] S.-M. Hung, S.N. Givigi, A. Noureldin, A dyna-q (λ) approach to flocking with fixed-wing uavs in a stochastic environment, in: 2015 IEEE International Conference on Systems, Man, and Cybernetics (SMC), Kowloon, 2015, pp. 1918–1923.
[25] S.-M. Hung, S.N. Givigi, A q-learning approach to flocking with uavs in a stochastic environment, IEEE Trans. Cybern. 47 (1) (2017) 186–197.
[26] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
[27] C. Speck, D.J. Bucci, Distributed uav swarm formation control via object-focused, multi-objective sarsa, in: 2018 Annual American Control Conference (ACC), Milwaukee, WI, USA, 2018, pp. 6596–6601.
[28] C. Yan, X. Xiang, C. Wang, Fixed-wing uavs flocking in continuous spaces: a deep reinforcement learning approach, Robot. Auton. Syst. 131 (2020) 103594.
[29] C. Yan, C. Wang, X. Xiang, Z. Lan, Y. Jiang, Deep reinforcement learning of collision-free flocking policies for multiple fixed-wing uavs using local situation maps, IEEE Trans. Ind. Inform. 18 (2) (2022) 1260–1270.
[30] M. Samvelyan, T. Rashid, C. Schroeder de Witt, G. Farquhar, N. Nardelli, T.G. Rudner, C.-M. Hung, P.H. Torr, J. Foerster, S. Whiteson, The starcraft multi-agent challenge, in: Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems, 2019, pp. 2186–2188.
[31] X. Wang, Y. Chen, W. Zhu, A survey on curriculum learning, IEEE Trans. Pattern Anal. Mach. Intell. 44 (9) (2022) 4555–4576.
[32] S.H. Semnani, H. Liu, M. Everett, A. de Ruiter, J.P. How, Multi-agent motion planning for dense and dynamic environments via deep reinforcement learning, IEEE Robot. Autom. Lett. 5 (2) (2020) 3221–3226.
[33] W. Wang, T. Yang, Y. Liu, J. Hao, X. Hao, Y. Hu, Y. Chen, C. Fan, Y. Gao, From few to more: large-scale dynamic multiagent curriculum learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, 2020, pp. 7293–7300.
[34] Q. Long, Z. Zhou, A. Gupta, F. Fang, Y. Wu, X. Wang, Evolutionary population curriculum for scaling multi-agent reinforcement learning, in: International Conference on Learning Representations (ICLR), 2020.
[35] F.A. Oliehoek, C. Amato, A Concise Introduction to Decentralized POMDPs, Springer, 2016.
[36] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, 2017, pp. 6382–6393.
[37] J.K. Gupta, M. Egorov, M. Kochenderfer, Cooperative multi-agent control using deep reinforcement learning, in: International Conference on Autonomous Agents and Multiagent Systems (AAMAS), Sao Paulo, Brazil, 2017, pp. 66–83.
[38] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.
[39] Z. Sui, Z. Pu, J. Yi, S. Wu, Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration, IEEE Trans. Neural Netw. Learn. Syst. 32 (6) (2021) 2358–2372.
[40] C. Yan, X. Xiang, C. Wang, Z. Lan, Flocking and collision avoidance for a dynamic squad of fixed-wing uavs using deep reinforcement learning, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2021, pp. 4738–4744.